

CS340 Assignment 1 Reflection

In this assignment, I learned a lot about how the xv6 kernel works, but I also faced some difficulties during the implementation. The hardest part for me was understanding trap handling in `usertrap()` and `kerneltrap()`. At first, I was not sure if I needed to put the tick-incrementing code in both functions. After carefully checking the code and thinking about how interrupts happen, I realized that timer interrupts can occur either when a process is running user-level code or when it is executing inside a system call in kernel mode. Another difficult task was using process locks (`p->lock`) correctly. When I wrote `sys_getpinfo()`, I had to make sure to acquire and release locks carefully so that process state and tick counters do not get corrupted by race conditions.

Through this project, I gained a much clearer understanding of core operating system concepts. I learned how system calls are connected step-by-step from user space to kernel space using files like `usys.pl`, `user.h`, `syscall.c`, and `sysproc.c`. It was interesting to see how `copyout()` is used to safely pass kernel data structures like `struct pinfo` to user space pointers. Furthermore, implementing the priority scheduler helped me understand how preemption works in practice. Writing the loop in `kernel/proc.c` showed me how high-priority processes (Priority 2) strictly run before low-priority ones (Priority 1), while processes at the same level share CPU time equally.

Overall, this assignment was very useful to practice real OS programming. Writing user-level test programs like `ps` and `schedtest` made it easy to verify my work. Seeing the Priority 2 child process finish its execution before the Priority 1 parent process proved that my priority scheduler and system calls were working correctly.